

FC7xxx CAN Integration Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0

Table of Contents

Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules.....	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory.....	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive Area.....	9
Chapter 5 Interrupt Service Routine (ISR).....	10
Chapter 6 Error Report.....	11
6.1 Det.....	11
6.2 Dem.....	12
Chapter 7 Function Calls.....	13
7.1 Function Calls during Startup	13
7.2 Function Calls during Shutdown	13
7.3 Function Calls during Wake-up	13
7.4 Function Calls during Runtime	13
Chapter 8 Other Requirements	14
8.1 Notification, Callback, Callout	14
8.2 Macros	14
Chapter 9 Integration Steps	15

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the CAN module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for CAN module
- Port: This module provides the port configurations if want output CAN signal through pin.
- Common: This module is the basic module which used to choose the chip.
- OS: This is used for CAN main function reference.
- EcuM: is required for selecting the reference to the wakeup source for every Can controller.
- Det: is required for signaling the default error detection (parameters out of range, null pointers, etc)

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for tracing Development errors.
- CanIf: This module provides the CanIf wakeup confirmation callback function.
- EcuM: This module provides the EcuM callback functions when wakeup support is enabled.

Module initialization dependency

- Mcu: To use the CAN module, the user needs to initialize the MCU module first.
- Port: To output CAN signal, the user needs to initialize PORT module first and assign the UART pins.

2.2 Files Required for Compile

CAN module files:

- _MCAL/Src/Can/Src/Can.c
- _MCAL/Src/Can/Src/Can_Hw.c
- _MCAL/Src/Can/Src/Can_Hal.c
- _MCAL/Src/Can/Src/Can_Irq.c
- _MCAL/Src/Can/include/Can.h
- _MCAL/Src/Can/include/Can_Hal.h
- _MCAL/Src/Can/include/Can_Hw.h
- _MCAL/Src/Can/include/Can_HwType.h
- _MCAL/Src/Can/include/Can_MemMap.h
- _MCAL/Src/Can/include/Fan_Reg.h
- _MCAL/Src/Can/include/Can_version.h
- _MCAL/Src/Can/include/Can_GeneralTypes.h
- _MCAL/Src/Can/include/Can_GeneralTypes_Cfg.h
- _MCAL_generate/src/Can_PBcfg.c
- _MCAL_generate/src/Can_Cfg.c
- _MCAL_generate/include/Can_PBcfg.h
- _MCAL_generate/include/Can_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:

- SchM_Can.h

CanIf module files:

- CanIf.h
- CanIf_Cbk.h

EcuM module files:

- EcuM_Cbk.h

2.3 Add Plug-ins

CAN module plug-ins are developed for EB tresos Studio, so, to use CAN plug-ins on the EB tresos Studio, the user needs to add the CAN module to the EB plug-ins folder first.

- 1) Copy the CAN module(_MCAL/EB_Plugins/eclipse/plugins/Can) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set the CAN module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the CAN module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
CAN_START_SEC_CONFIG_DATA_8 CAN_STOP_SEC_CONFIG_DATA_8 CAN_START_SEC_CONFIG_DATA_16 CAN_STOP_SEC_CONFIG_DATA_16 CAN_START_SEC_CONFIG_DATA_32 CAN_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code(data).
CAN_START_SEC_CONFIG_DATA_UNSPECIFIED CAN_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
CAN_START_SEC_CONST_BOOLEAN CAN_STOP_SEC_CONST_BOOLEAN CAN_START_SEC_CONST_8 CAN_STOP_SEC_CONST_8 CAN_START_SEC_CONST_16 CAN_STOP_SEC_CONST_16 CAN_START_SEC_CONST_32 CAN_STOP_SEC_CONST_32 CAN_START_SEC_CONST_UNSPECIFIED CAN_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
CAN_START_SEC_CODE CAN_STOP_SEC_CODE CAN_START_SEC_RAMCODE CAN_STOP_SEC_RAMCODE CAN_START_SEC_CODE_AC CAN_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
CAN_START_SEC_VAR_NO_INIT_BOOLEAN CAN_STOP_SEC_VAR_NO_INIT_BOOLEAN CAN_START_SEC_VAR_NO_INIT_8 CAN_STOP_SEC_VAR_NO_INIT_8 CAN_START_SEC_VAR_NO_INIT_16 CAN_STOP_SEC_VAR_NO_INIT_16 CAN_START_SEC_VAR_NO_INIT_32 CAN_STOP_SEC_VAR_NO_INIT_32 CAN_START_SEC_VAR_NO_INIT_UNSPECIFIED CAN_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
CAN_START_SEC_VAR_INIT_BOOLEAN CAN_STOP_SEC_VAR_INIT_BOOLEAN CAN_START_SEC_VAR_INIT_8 CAN_STOP_SEC_VAR_INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
CAN_START_SEC_VAR_INIT_16		startup code (data).
CAN_STOP_SEC_VAR_INIT_16		
CAN_START_SEC_VAR_INIT_32		
CAN_STOP_SEC_VAR_INIT_32		
CAN_START_SEC_VAR_INIT_UNSPECIFIED		
CAN_STOP_SEC_VAR_INIT_UNSPECIFIED		

Chapter 4 Exclusive Area

CAN module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the CAN driver:

- Can_Hal.c:
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_00 is used in Can_Hal_DisableControllerInterrupts.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_01 is used in Can_Hal_EnableControllerInterrupts.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_03 is used in Can_LL_GotoFreezeMode.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_04 is used in Can_LL_ResetController.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_05 is used in Can_LL_InitBaudrate.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_06 is used in Can_ML_SetControllerToStopMode.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_07 is used in Can_LL_DisableInterrupts.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_08 is used in Can_LL_EnableInterrupts.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_09 is used in Can_LL_SelectClockSource.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_11 is used in Can_LL_InitRxFifo.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_13 is used in Can_HL_SetControllerToStartMode.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_14 is used in Can_LL_RxDataReadyHandler.
 - SchM_Enter_Can_CAN_EXCLUSIVE_AREA_15 is used in Can_LL_RxDataReadyHandler.

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
FLEXCAN0	Can_IsrCAN0_All	84
FLEXCAN1	Can_IsrCAN1_All	85
FLEXCAN2	Can_IsrCAN2_All	86
FLEXCAN3	Can_IsrCAN3_All	87
FLEXCAN4	Can_IsrCAN4_All	88
FLEXCAN5	Can_IsrCAN5_All	89
FLEXCAN6	Can_IsrCAN6_All	90
FLEXCAN7	Can_IsrCAN7_All	91

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Can_Init	CAN_E_TRANSITION; CAN_E_PARAM_POINTER;
Can_GetVersionInfo	CAN_E_PARAM_POINTER;
Can_DelInit	CAN_E_TRANSITION
Can_SetBaudrate	CAN_E_UNINIT; CAN_E_PARAM_BAUDRATE; CAN_E_PARAM_CONTROLLER;
Can_SetControllerMode	CAN_E_UNINIT; CAN_E_PARAM_CONTROLLER; CAN_E_TRANSITION;
Can_DisableControllerInterrupts	CAN_E_UNINIT; CAN_E_PARAM_CONTROLLER;
Can_EnableControllerInterrupts	CAN_E_UNINIT; CAN_E_PARAM_CONTROLLER;
Can_CheckWakeup	CAN_E_UNINIT; CAN_E_PARAM_CONTROLLER;
Can_GetControllerErrorState	CAN_E_UNINIT; CAN_E_PARAM_CONTROLLER;
Can_GetControllerMode	CAN_E_UNINIT; CAN_E_PARAM_CONTROLLER; CAN_E_PARAM_POINTER;
Can_Write	CAN_E_UNINIT; CAN_E_PARAM_DATA_LENGTH; CAN_E_PARAM_POINTER;
Can_MainFunction_Write	CAN_E_UNINIT;
Can_MainFunction_Write_0	CAN_E_UNINIT;
Can_MainFunction_Write_1	CAN_E_UNINIT;
Can_MainFunction_Write_2	CAN_E_UNINIT;
Can_MainFunction_Write_3	CAN_E_UNINIT;
Can_MainFunction_Write_4	CAN_E_UNINIT;
Can_MainFunction_Read	CAN_E_UNINIT;
Can_MainFunction_Read_0	CAN_E_UNINIT;
Can_MainFunction_Read_1	CAN_E_UNINIT;
Can_MainFunction_Read_2	CAN_E_UNINIT;
Can_MainFunction_Read_3	CAN_E_UNINIT;
Can_MainFunction_Read_4	CAN_E_UNINIT;
Can_MainFunction_BusOff	CAN_E_UNINIT;

Can_MainFunction_Wakeup	CAN_E_UNINIT;
Can_MainFunction_Mode	CAN_E_UNINIT;

6.2 Dem

None

Chapter 7 Function Calls

7.1 Function Calls during Startup

CAN shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is `Can_Init()`. The MCU module should be initialized before the CAN is initialized. The CAN module shall be initialized by `Can_Init(<&Can_Configuration>)` service call during the start-up before the CAN peripherals are used. Please note that GPIO pins used for connection of CAN physical layer have to be properly assigned to the desired module prior to the CAN initialization: so the MCU and PORT modules shall be initialized before CAN is initialized. After the CAN module is initialized each CAN controller has to be initialized as well before using it. This is also done by the `Can_Init(<&Can_Configuration>)` service.

7.2 Function Calls during Shutdown

- **Freeze Mode**

This low power mode is entered when the HALT and FRZ bits in the MCR Register are asserted. Module ignores the Rx input pin and drives the Tx pin as recessive, stops the prescaler, thus halting all CAN protocol activities and grants write access to the Error Counters Register (ECR), which is read-only in other modes. Exit from this mode is done by negating the FRZ and HALT bits in the MCR Register or when the MCU is removed from Debug Mode.

- **Module Disable Mode**

This low power mode is entered when the MDIS bit in the MCR Register is asserted. Module shuts down the clocks to the CAN Protocol Interface and Message Buffer Management sub-modules. Exit from this mode is done by negating the MDIS bit in the MCR Register.

- **Stop Mode**

This low power mode is entered when Stop Mode is requested at the MCU level. When in Stop Mode, the module puts itself in an inactive state and then informs the CPU that the clocks can be shut down globally. Exit from this mode happens when the Stop Mode request is removed or when activity is detected on the CAN bus and the Self Wake Up mechanism is enabled.

7.3 Function Calls during Wake-up

The controller can be woken up by a message when it is in Sleep mode only if the interrupt for wakeup is enabled or the self wakeup mechanism is enabled. When wakeup events are detected by polling, the ECU State Manager will cyclically call `Can_CheckWakeup`. CAN stack can be changed from SLEEP mode to STOP mode by calling the `Can_SetControllerMode(CAN_T_WAKEUP)` service call.

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Notification, Callback, Callout

- Notification

CAN driver shall use the following APIs provided by ICU driver, to enable and disable the wakeup event notification:

- 1) Icu_EnableNotification.
- 2) Icu_DisableNotification.

- Callback

There are no callback functions within the CAN driver.

- Callout

CAN module supports optional L-PDU callouts on every reception of a L-PDU. The L-PDU-Callout API shall be defined as:

```
FUNC (boolean, COM_APPL_CODE) <LPDU_CalloutName>(uint8 Hrh, Can_IdType CanId, uint8 CanDlc, const uint8 *CanSduPtr)
```

The CAN Driver uses these mandatory callouts functions which have to be provided by the respective can stack:

CanIf_ControllerBusOff, CanIf_ControllerModeIndication, CanIf_RxIndication, CanIf_TxConfirmation, GetCounterValue.

These functions are optional:

CanIf_CurrentComConfiguration, CanIf_TriggerTransmit, Dem_ReportErrorStatus, Det_ReportError, EcuM_CheckWakeup, EcuM_SetWakeupEvent.

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:

AUTOSAR_OS_NOT_USED needs to be defined.

- If OS is used:

Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration Steps

- 1) Configure the CAN module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the CAN module with dependent modules.